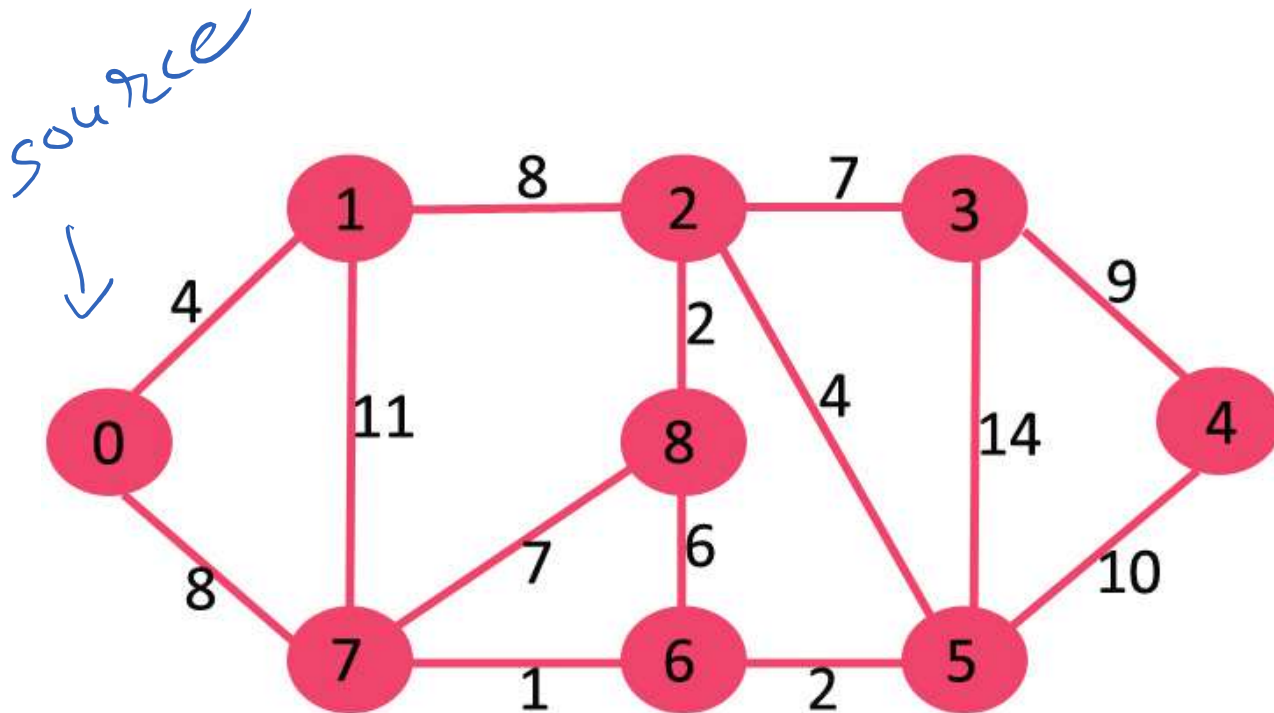
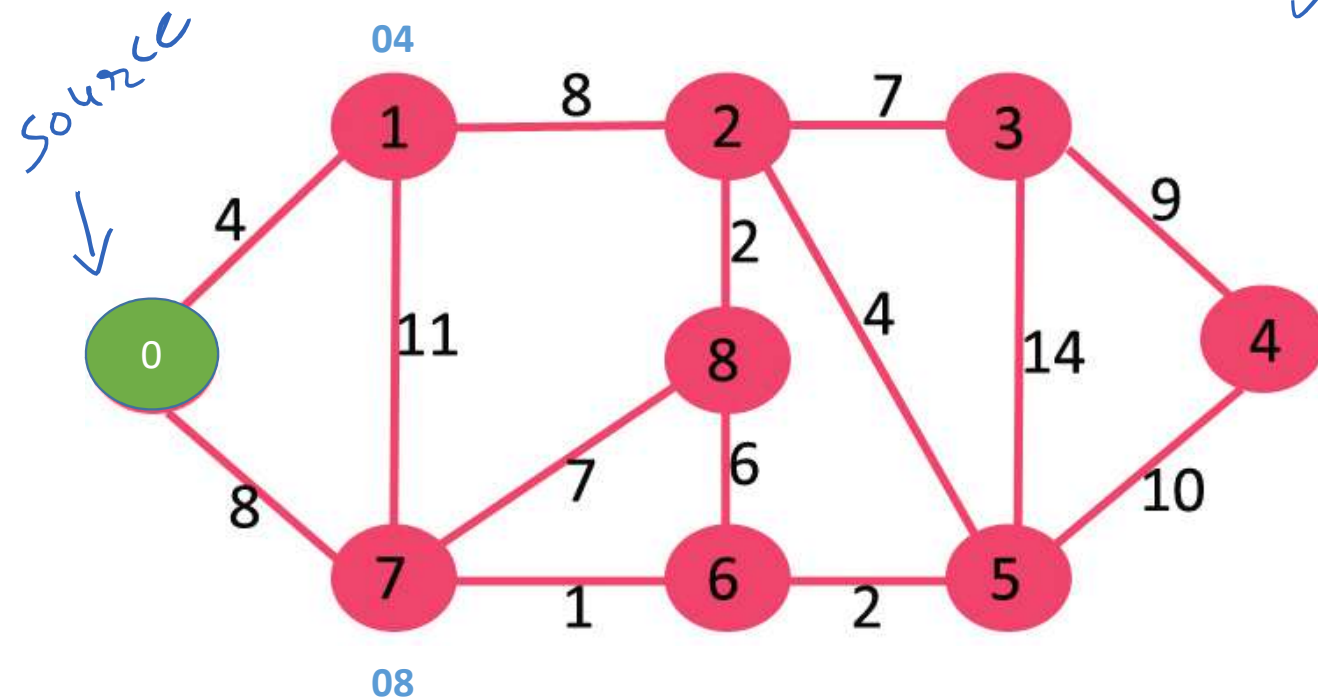


Di'jkstra Algorithm (Shortest Path)



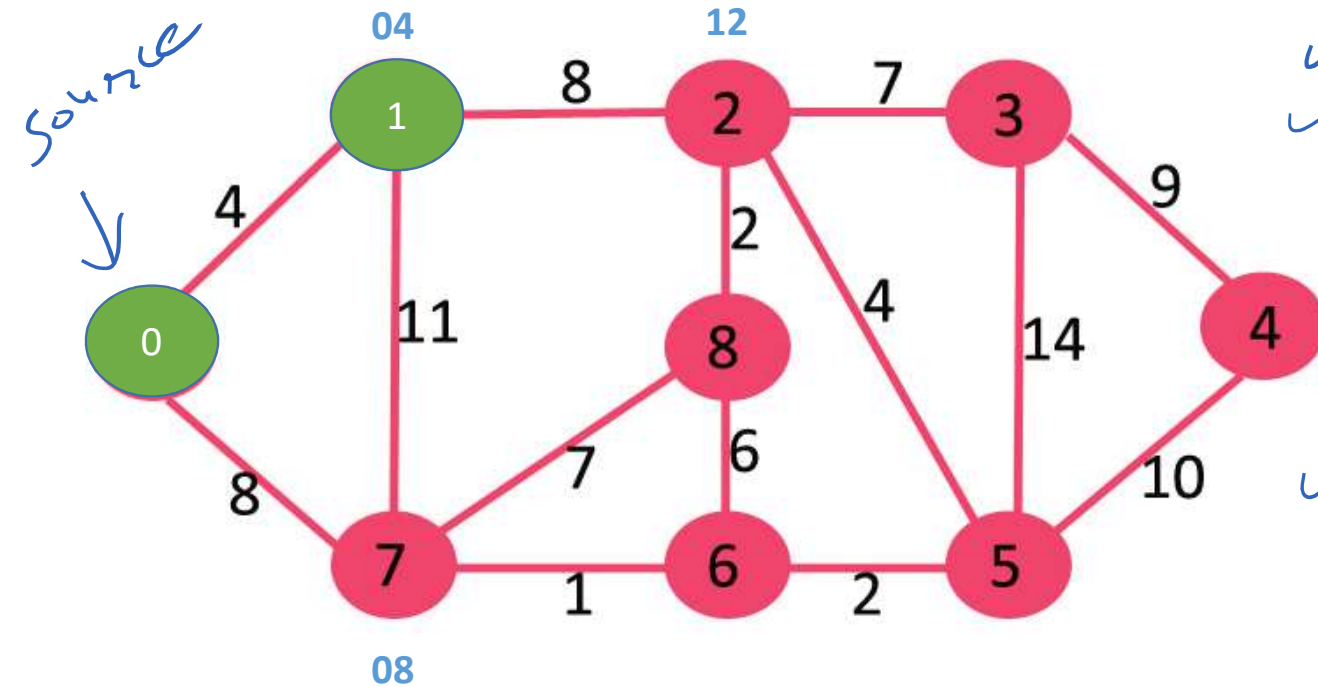
Node	Distance	Visited	Previous
0	0 ✓	F	-1
1	INF	F	-1
2	INF	F	-1
3	INF	F	-1
4	INF	F	-1
5	INF	F	-1
6	INF	F	-1
7	INF	F	-1
8	INF	F	-1

Di'jkstra Algorithm (Shortest Path)



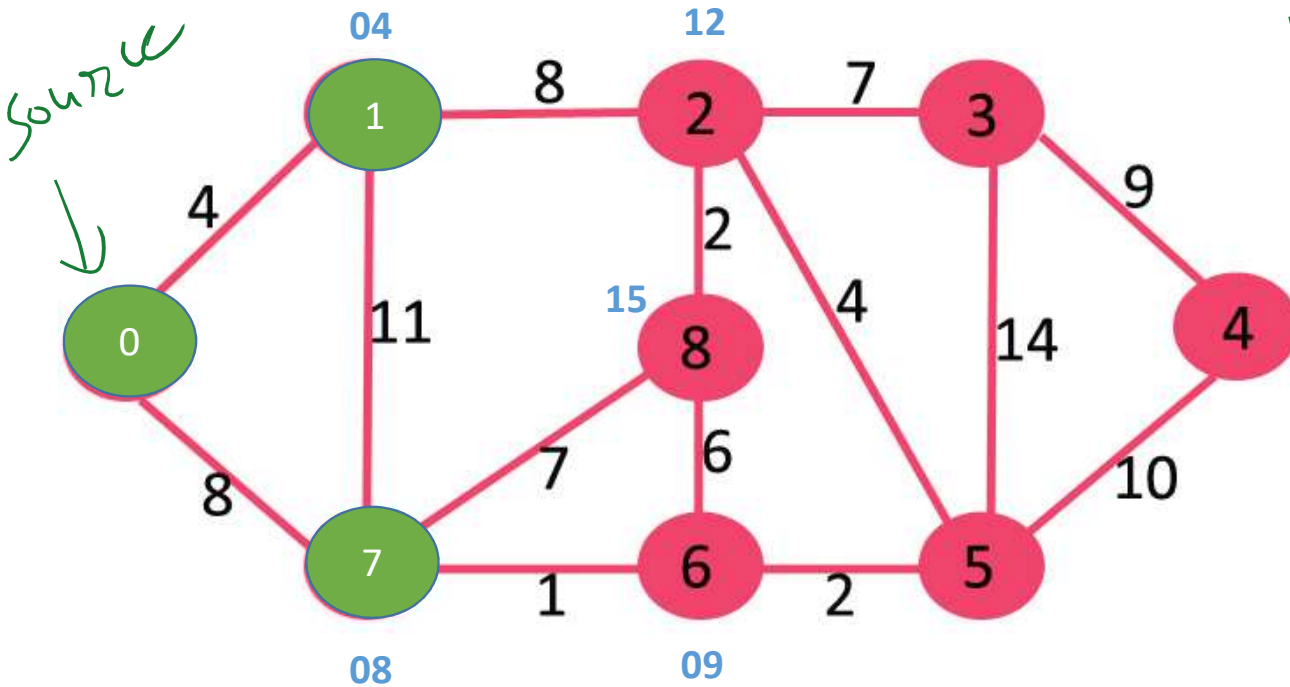
Node	Distance	Visited	Previous
0	0	T ✓	-1 ✓
1	04 ✓	F	0 ✓
2	INF	F	-1
3	INF	F	-1
4	INF	F	-1
5	INF	F	-1
6	INF	F	-1
7	08 ✓	F	0 ✓
8	INF	F	-1

Di'jkstra Algorithm (Shortest Path)



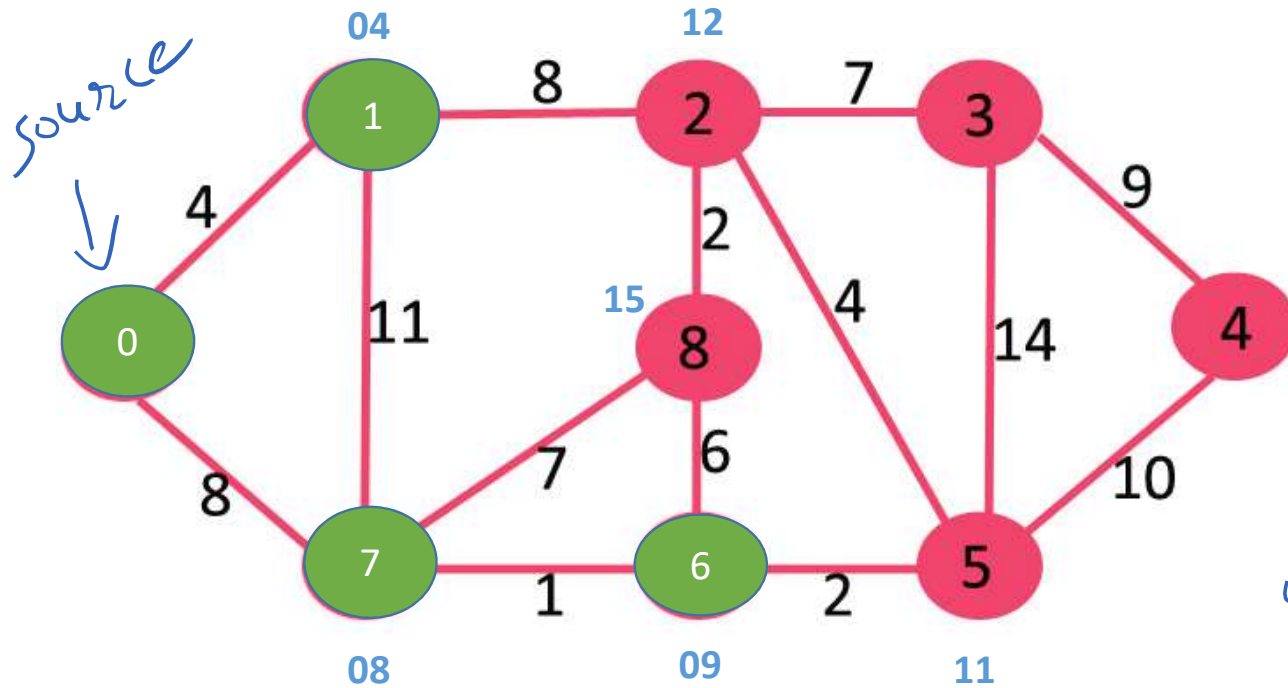
Node	Distance	Visited	Previous
✓ 0	0	T	-1
✓ 1	04	T	0
✓ 2	12 ✓	F	1 ✓
3	INF	F	-1
4	INF	F	-1
5	INF	F	-1
6	INF	F	-1
✓ 7	08 ✓	F ✓	0 ✓
8	INF	F	-1

Di'jkstra Algorithm (Shortest Path)



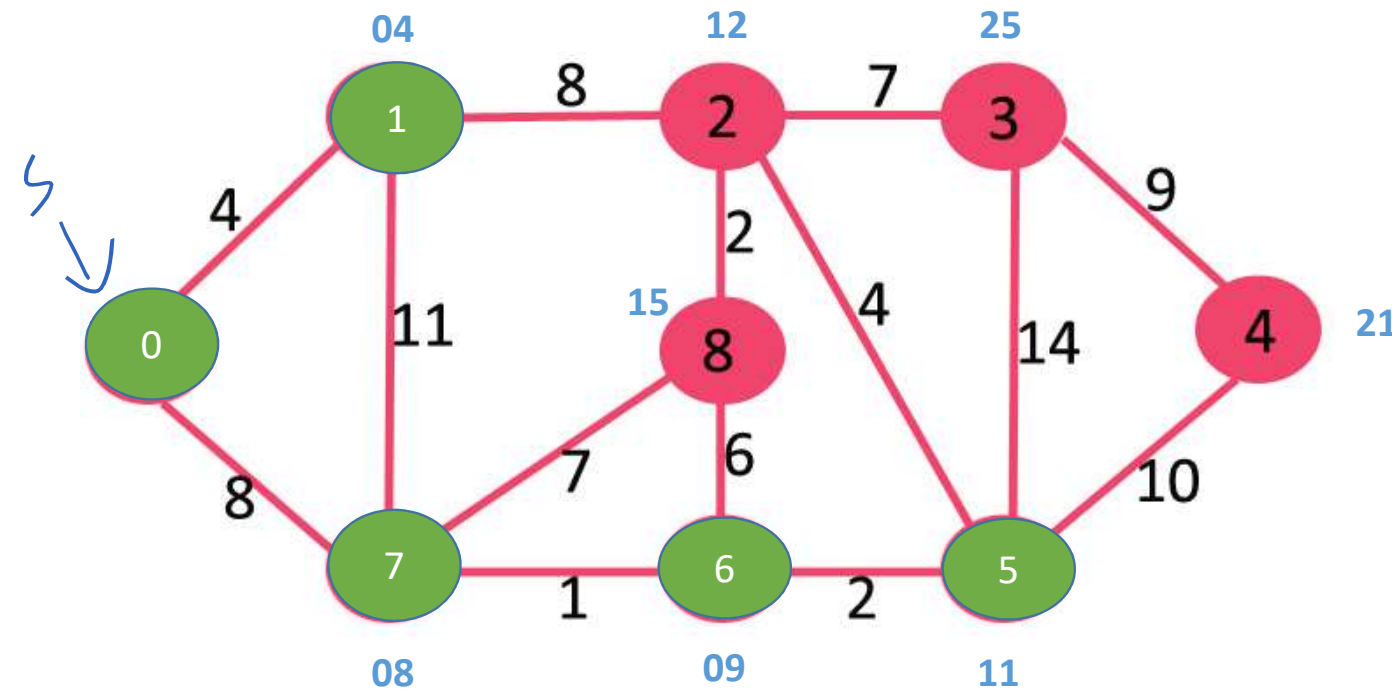
Node	Distance	Visited	Previous
0	0	T	-1
1	04	T	0
2	12	F	1
3	INF	F	-1
4	INF	F	-1
5	INF	F	-1
6	09	F	7
7	08	T	0
8	15	F	7

Di'jkstra Algorithm (Shortest Path)



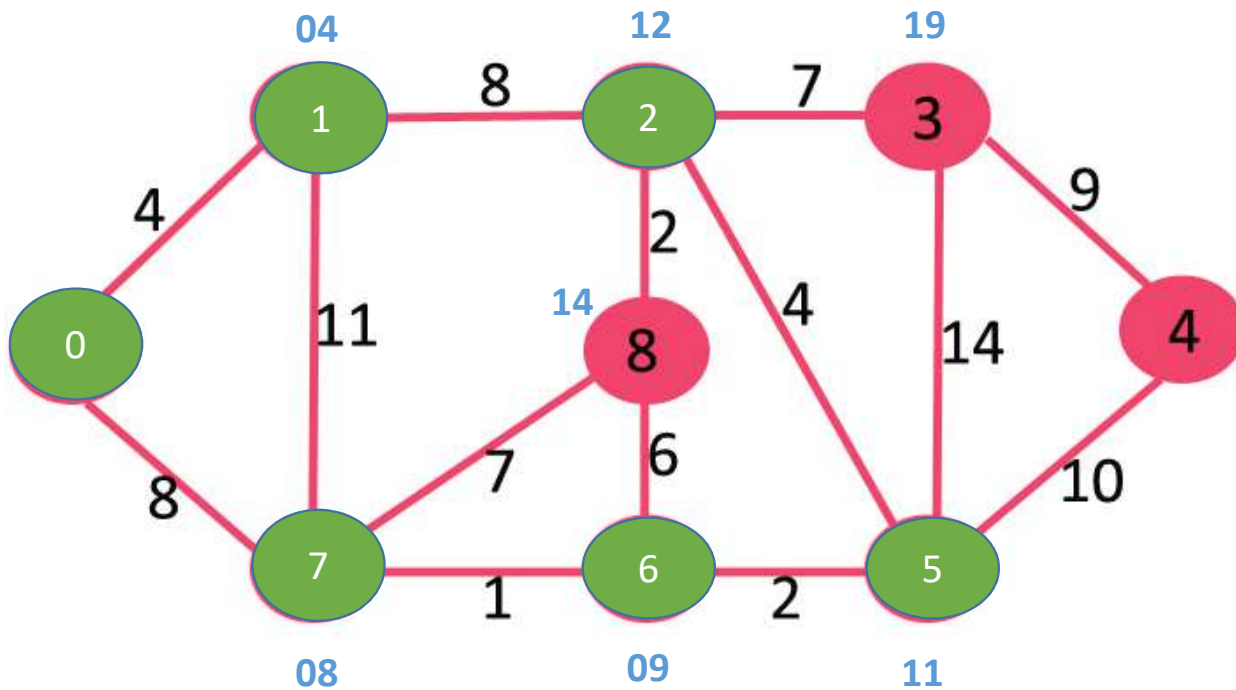
Node	Distance	Visited	Previous
✓ 0	0	T ✓	-1
✓ 1	04	T ✓	0
2	12	F	1
3	INF	F	-1
4	INF	F	-1
✓ 5	11 ✓	F	6 ✓
✓ 6	09	T ✓	7
✓ 7	08	T ✓	0
✓ 8	15	F	7

Di'jkstra Algorithm (Shortest Path)



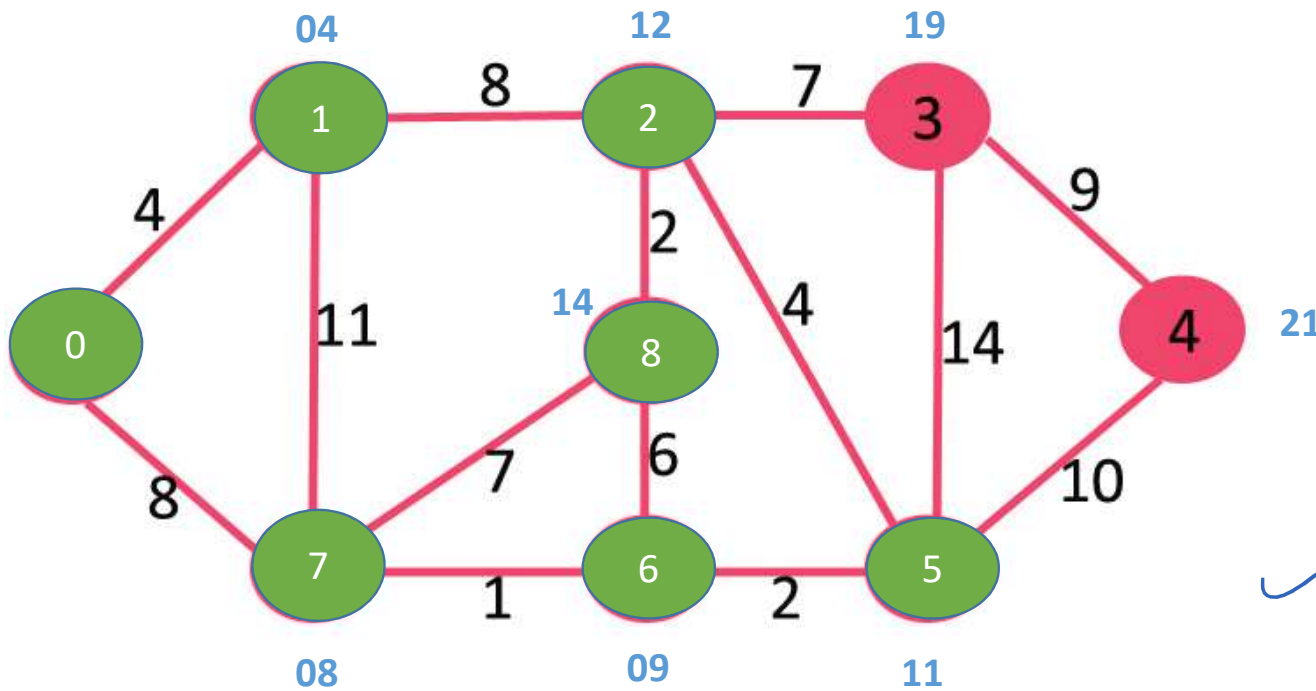
Node	Distance	Visited	Previous
0	0	T	-1
1	04	T	0
2	12	F	1
3	25	F	5 ✓
4	21	F	5 ✓
5	11	T ✓	6
6	09	T	7
7	08	T	0
8	15	F	7

Di'jkstra Algorithm (Shortest Path)



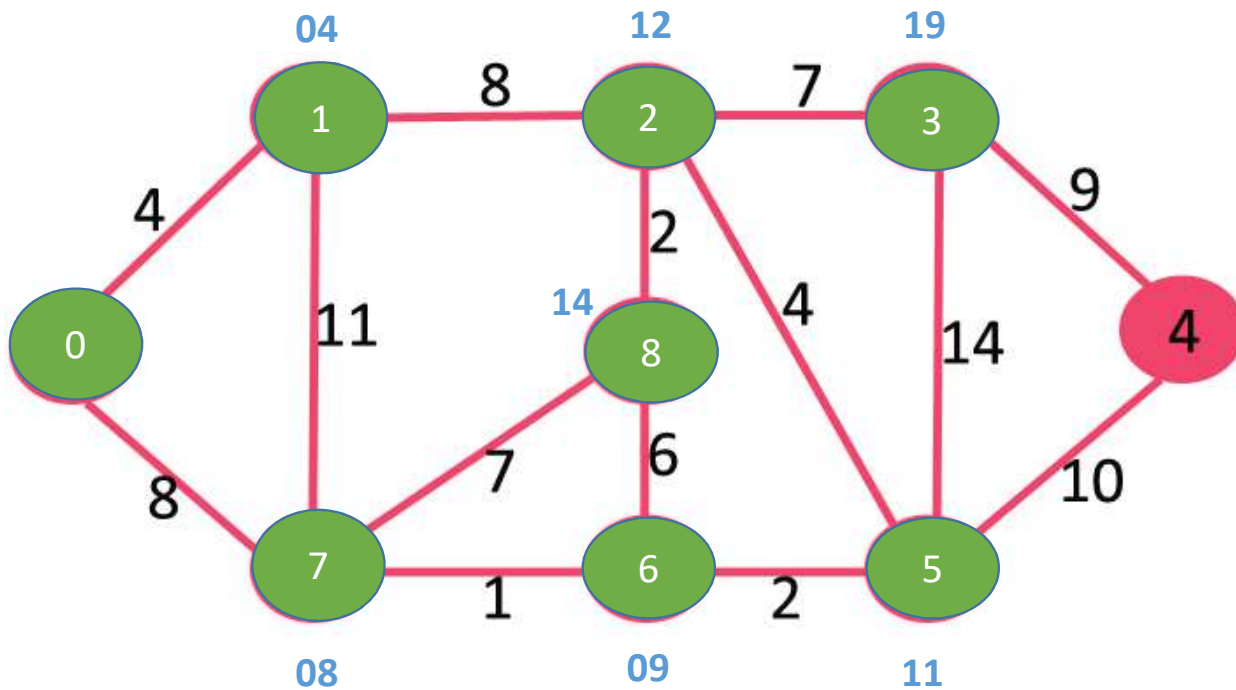
Node	Distance	Visited	Previous
✓ 0	0	T	-1
✓ 1	04	T	0
✓ 2	12	T ✓	1
✓ 3	19 ✓	F	2 ✓
21 4	21	F	5
5	11	T	6
6	09	T	7
7	08	T	0
✓ 8	14 ✓	F	2 ✓

Di'jkstra Algorithm (Shortest Path)



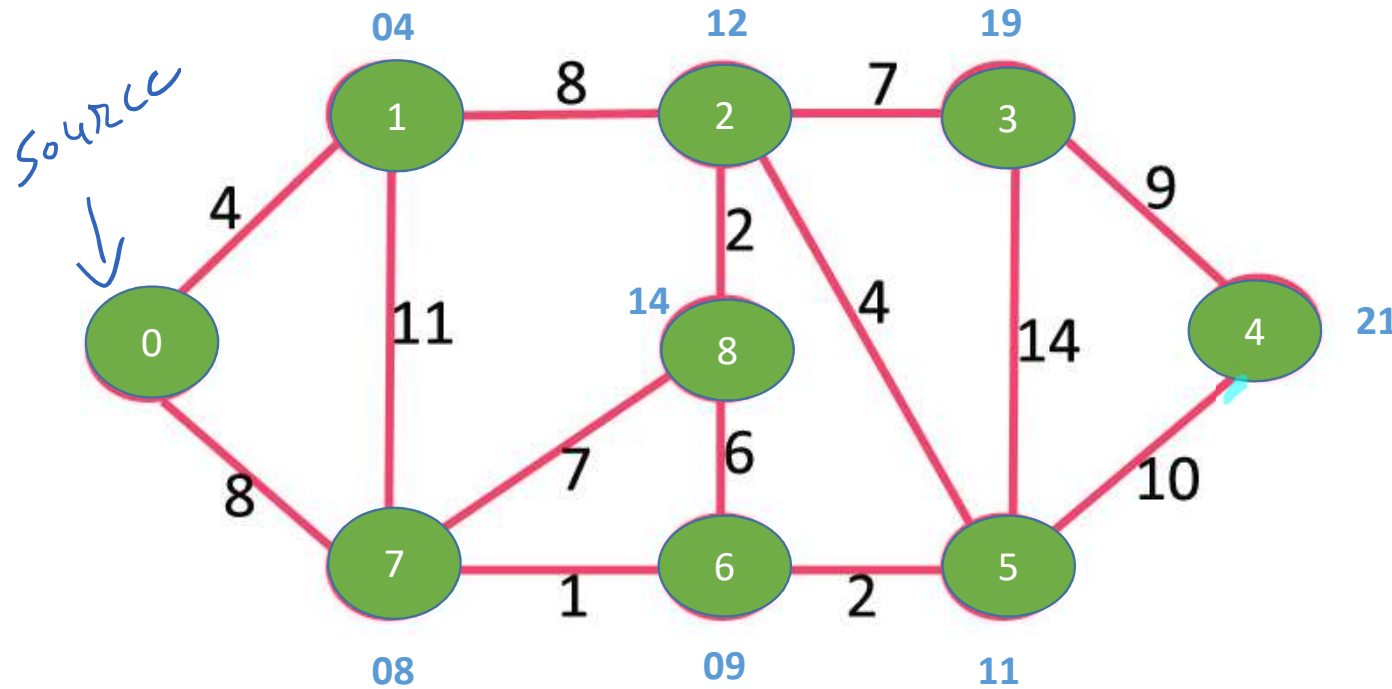
Node	Distance	Visited	Previous
0	0	T	-1
1	04	T	0
2	12	T	1
3	19	F	2
4	21	F	5
5	11	T	6
6	09	T	7
7	08	T	0
8	14	T ✓	2

Di'jkstra Algorithm (Shortest Path)



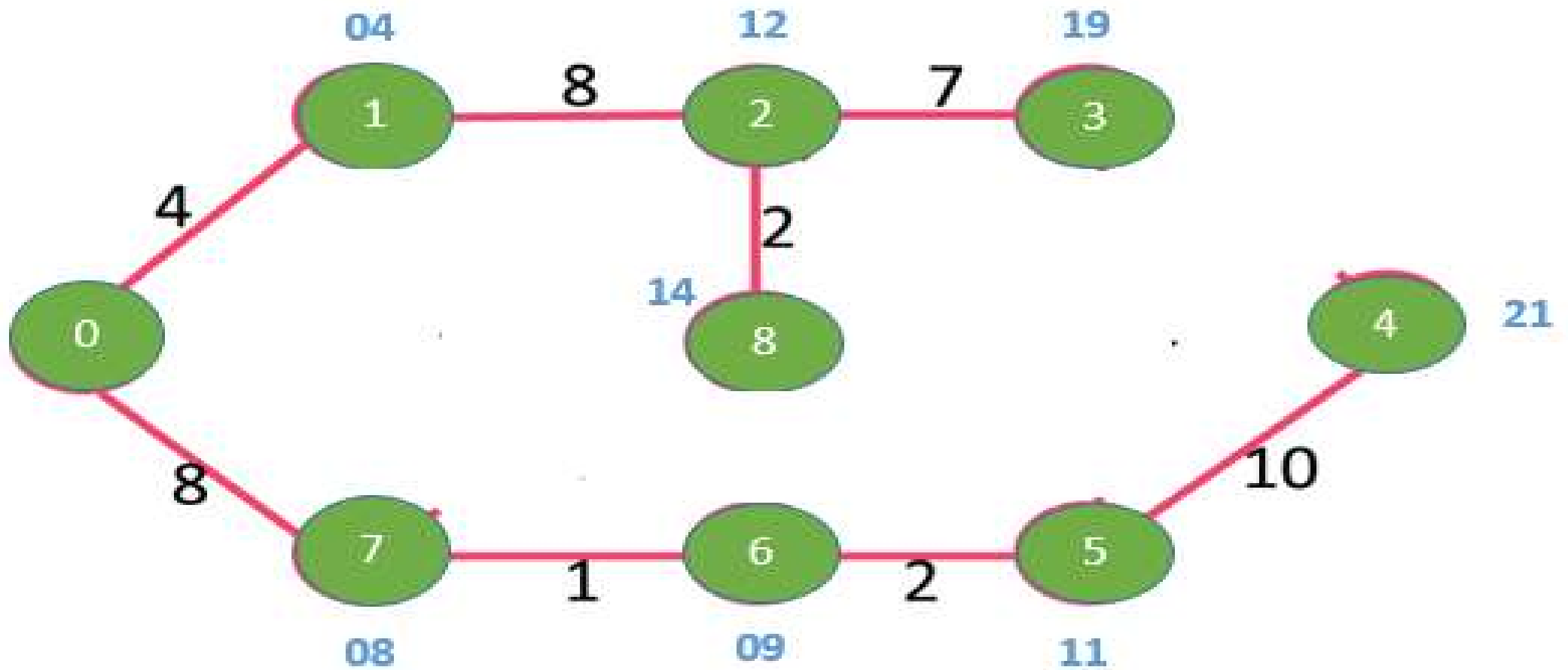
Node	Distance	Visited	Previous
0	0	T	-1
1	04	T	0
2	12	T	1
3	19	T	2
4	21	F	5
5	11	T	6
6	09	T	7
7	08	T	0
8	14	T	2

Di'jkstra Algorithm (Shortest Path)



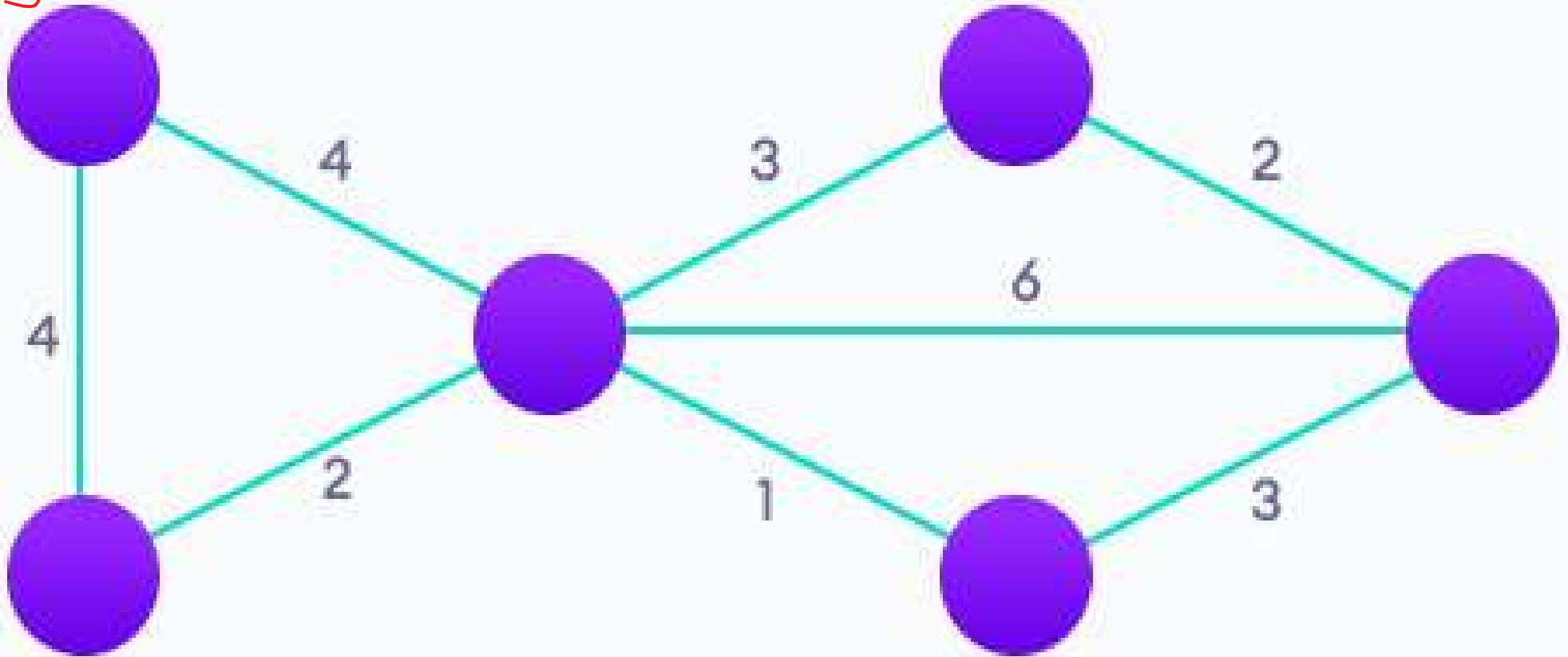
Node	Distance	Visited	Previous
0	0	T	-1
1	04	T	0
2	12	T	1
3	19	T	2
4	21	T	5
5	11	T	6
6	09	T	7
7	08	T	0
8	14	T	2

Di'jkstra Algorithm (Shortest Path Tree)



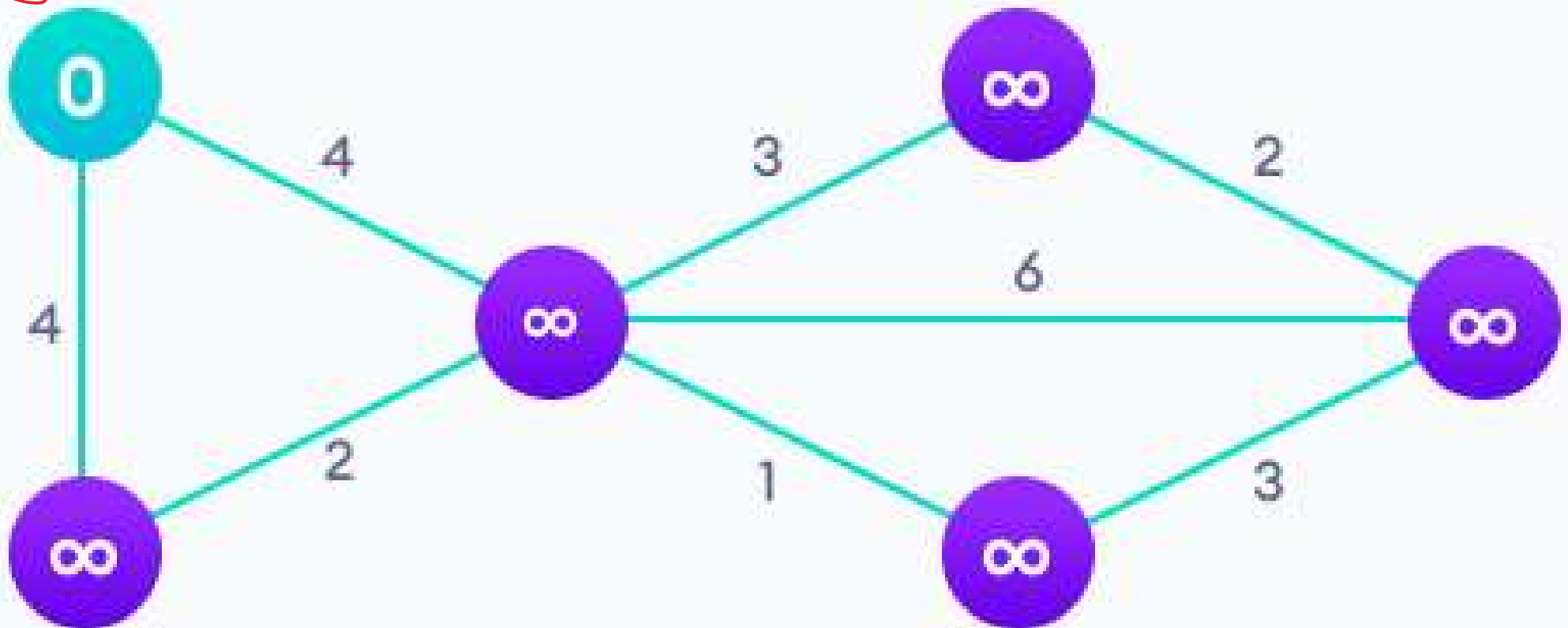
Di'jkstra Algorithm (Shortest Path)

source

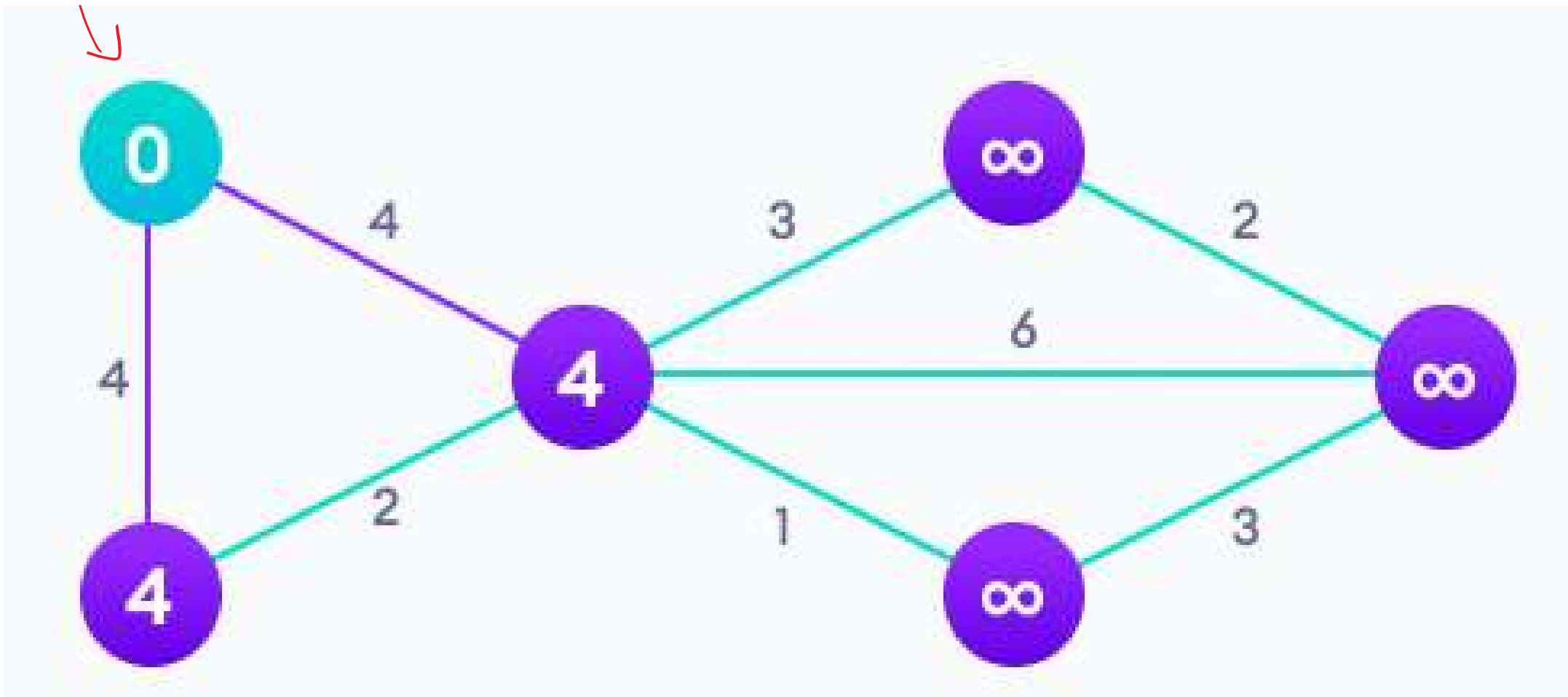


Di'jkstra Algorithm (Shortest Path)

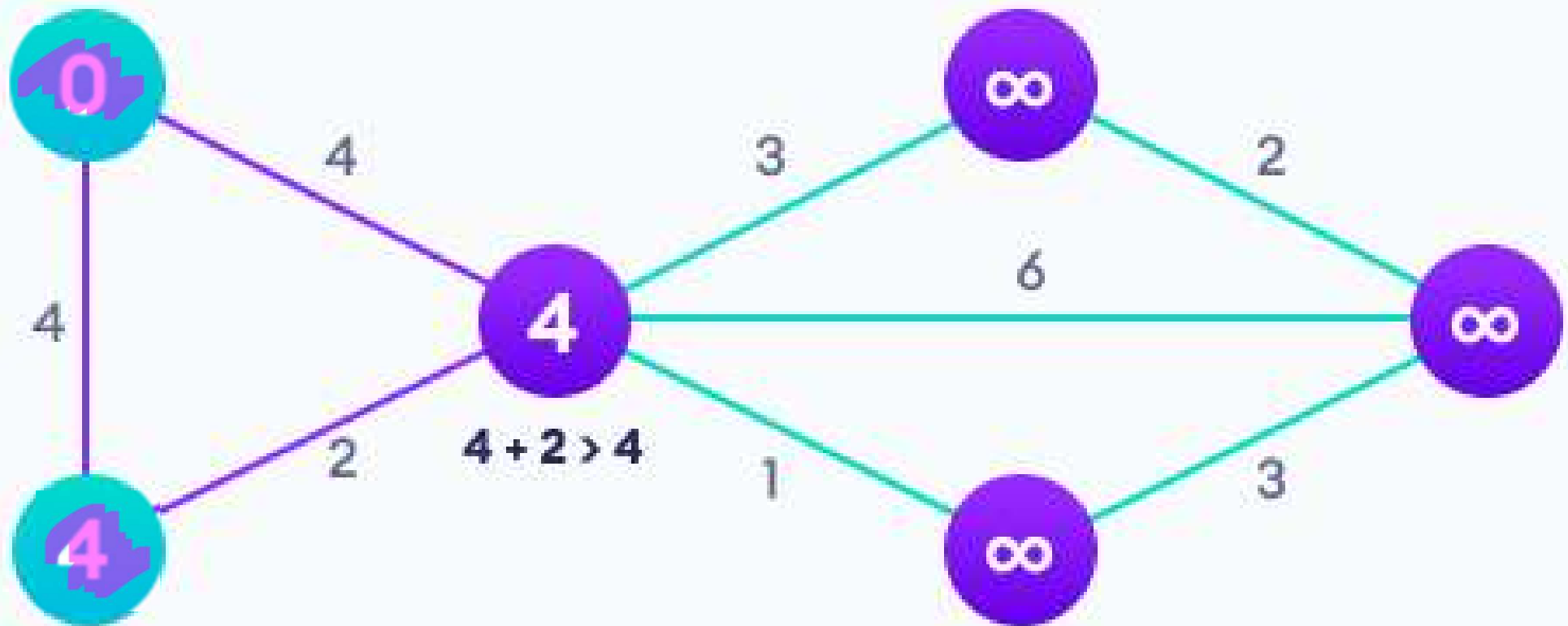
Source
↓



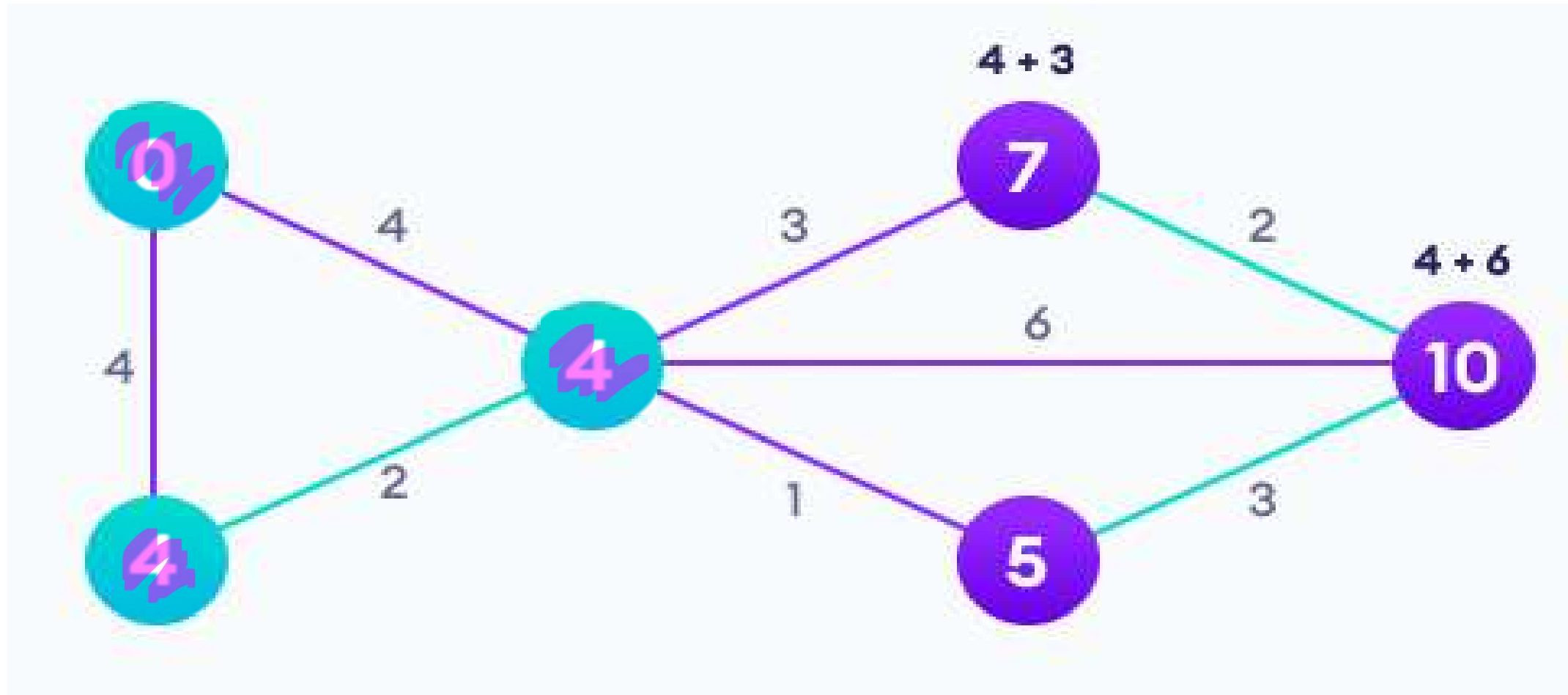
Di'jkstra Algorithm (Shortest Path)



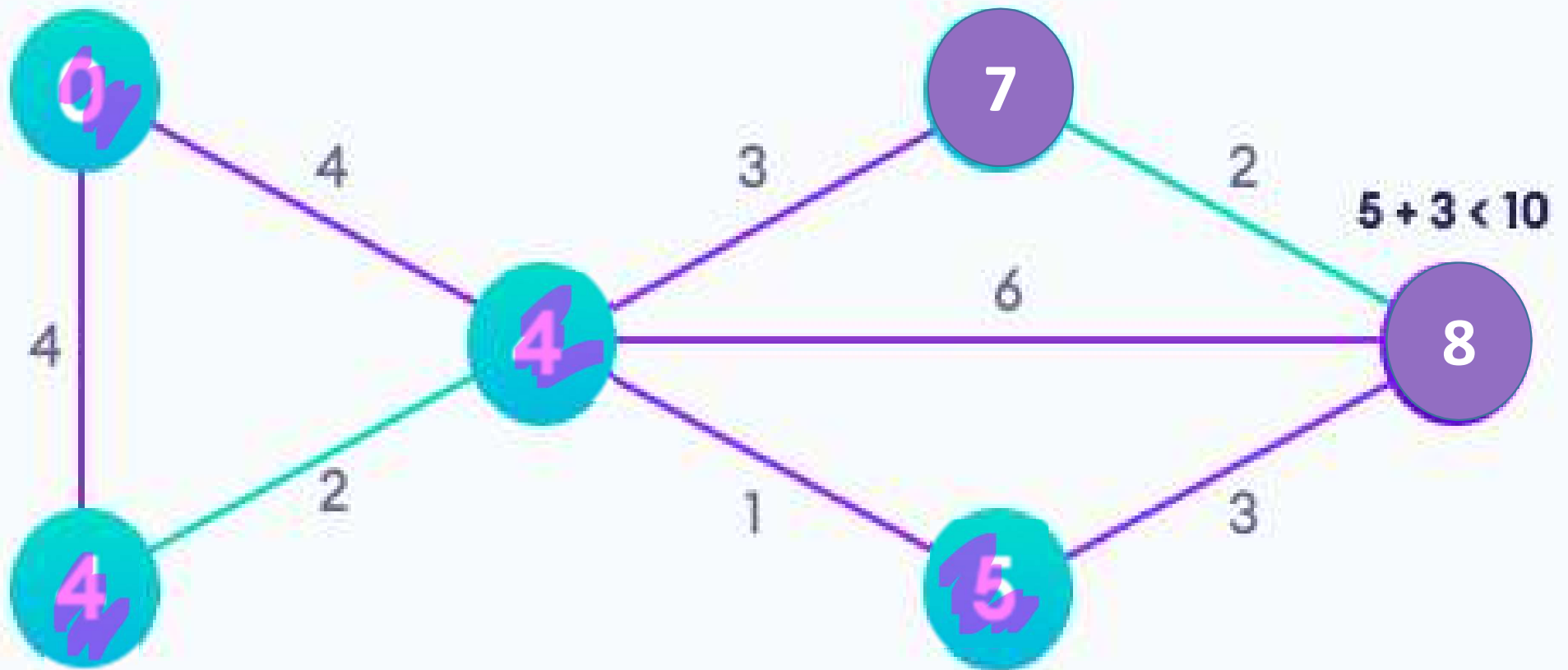
Di'jkstra Algorithm (Shortest Path)



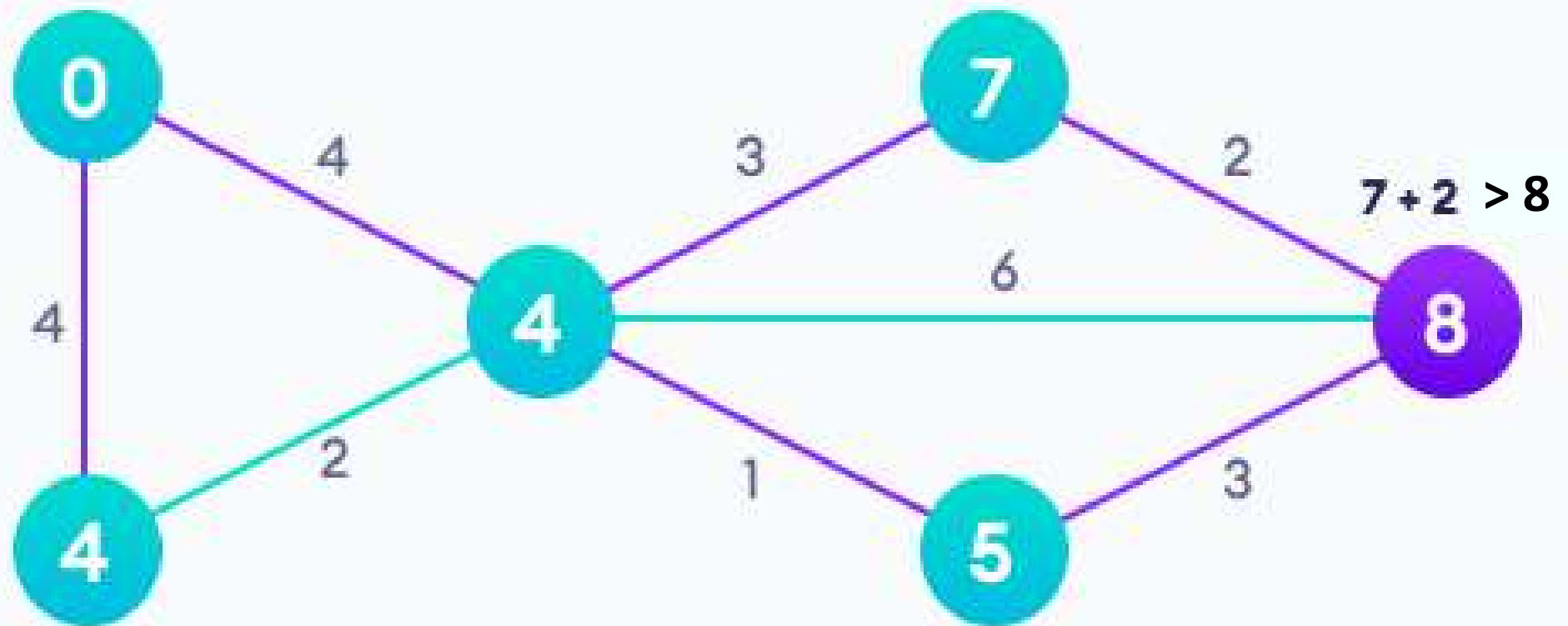
Di'jkstra Algorithm (Shortest Path)



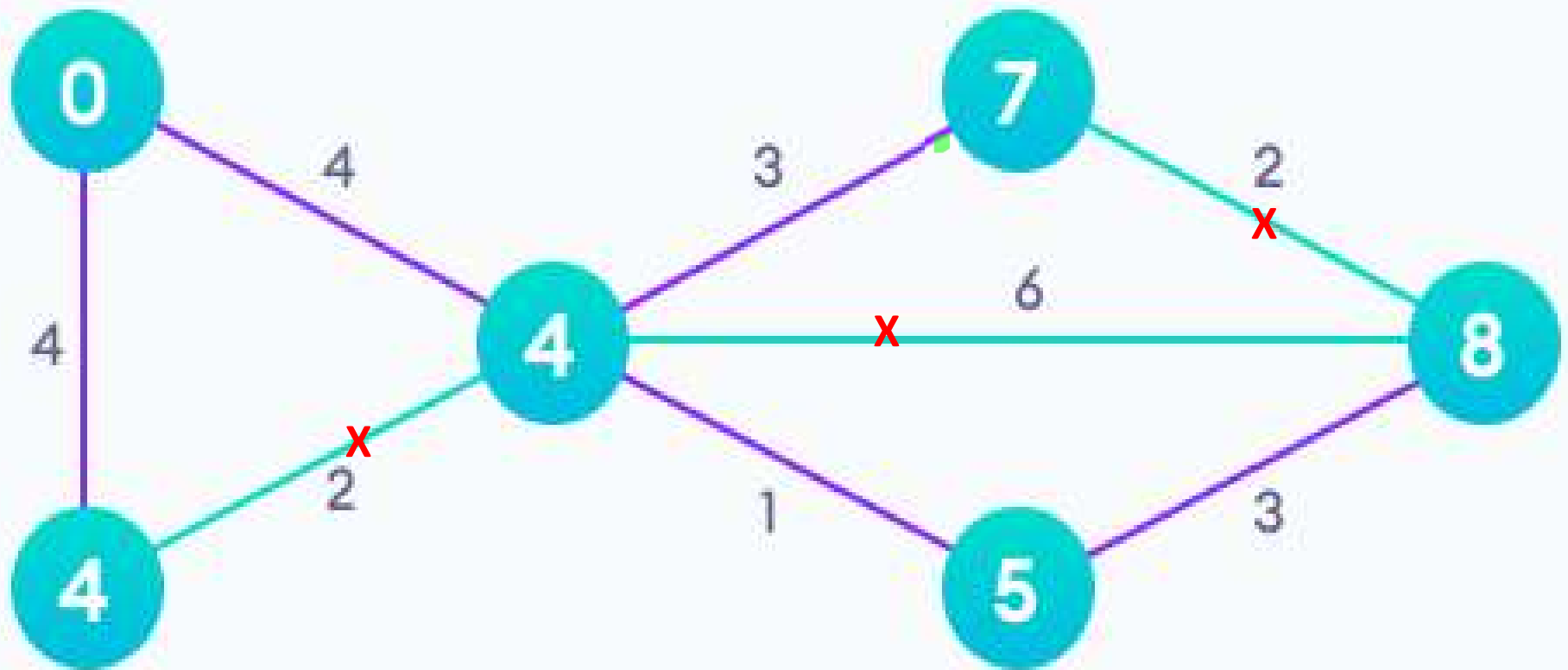
Di'jkstra Algorithm (Shortest Path)



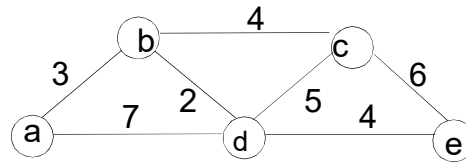
Di'jkstra Algorithm (Shortest Path)



Di'jkstra Algorithm (Shortest Path)



Example : 3

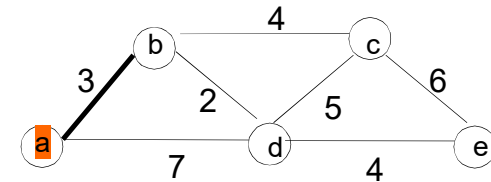


Tree vertices

Remaining vertices

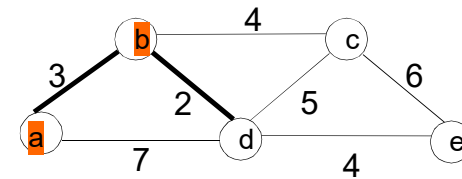
$a(-,0)$

$b(a,3)$ $c(-,\infty)$ $d(a,7)$ $e(-,\infty)$



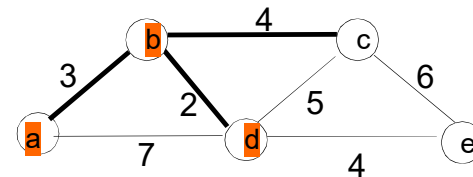
$b(a,3)$

$c(b,3+4)$ $d(b,3+2)$ $e(-,\infty)$



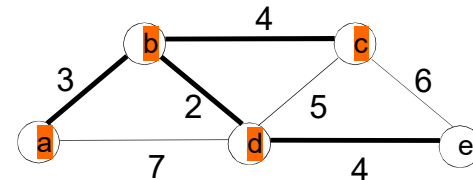
$d(b,5)$

$c(b,7)$ $e(d,5+4)$



$c(b,7)$

$e(d,9)$



$e(d,9)$

Algorithm

```
1:  function Dijkstra(Graph, source):
2:      for each vertex v in Graph:                // Initialization
3:          dist[v] := infinity                    // initial distance from source to vertex v is set to infinite
4:          previous[v] := undefined               // Previous node in optimal path from source
5:      dist[source] := 0                          // Distance from source to source
6:      Q := the set of all nodes in Graph          // all nodes in the graph are unoptimized - thus are in Q
7:      while Q is not empty:                      // main loop
8:          u := node in Q with smallest dist[ ]
9:          remove u from Q
10:         for each neighbor v of u:              // where v has not yet been removed from Q.
11:             alt := dist[u] + dist_between(u, v)
12:             if alt < dist[v]                   // Relax (u,v)
13:                 dist[v] := alt
14:                 previous[v] := u
15:     return previous[ ]
```

Notes on Dijkstra's algorithm

- Doesn't work for graphs with negative weights
- Applicable to both undirected and directed graphs
- Efficiency
 - $O(|V|^2)$ for graphs represented by weight matrix and array/ LINKED LIST implementation of priority queue
 - $O(|E| \log |V|)$ for graphs represented by adj. lists and min-heap implementation of priority queue